

EE3621: Digital Signal Processing

Lecture 14a: Visual Concepts — FFT Algorithms & Duality of Aliasing (III B.Tech EEE)

September 9, 2026

1. LEARNING OBJECTIVES

By the end of this supplementary lecture, students will be able to:

1. **Understand** the intuitive mechanism of Time-Aliasing during FFT circular convolution.
 2. **Visualize** the Overlap-Save block processing pipeline without heavy mathematics.
 3. **Compare** the structural differences between Overlap-Add and Overlap-Save using a visual ‘Triangle’ approach.
-

2. THE INTUITION: WHY OVERLAP-SAVE?

In Lecture 13, we explored the **Overlap-Add (OLA)** method. In OLA, we chopped our infinite audio stream into independent blocks, padded them with zeros, and used the FFT. Because of the zeros, the linear convolution created a ‘tail’ that spilled over the block length. We had to physically **add** those overlapping tails together to reconstruct the signal.

The Overlap-Save (OLS) Alternative: What if we want to avoid that final addition step? (Addition takes extra CPU cycles and memory management).

- What happens if we just feed full blocks of data into the FFT without padding them with zeros?
- **The Problem:** The FFT inherently computes **Circular Convolution**. If the signal isn’t padded, the ‘tail’ of the convolution doesn’t have empty space to stretch into. Instead, it wraps around the circle and crashes into the beginning of your block. This corruption is called **Time Aliasing**.
- **The Brilliant Solution (The ‘Trash’ Method):** If a filter has *taps*, *we know exactly how much of the signal is corrupted*. We simply throw away the first t samples and keep the rest.

Instead of trying to prevent the corruption, Overlap-Save deliberately lets it happen! We intentionally overlap our input blocks so that the FFT corrupts old data we already processed. Then, we simply throw the corrupted t samples in the trash and save the good ones.

3. VISUALIZING FFT ALIASING & RECONSTRUCTION

Just as making time discrete causes frequency to become periodic (requiring an anti-aliasing filter and a reconstruction filter to isolate the baseband triangle), making frequency discrete (which the FFT does) causes time to become periodic!

If we use an FFT size that is too small for our linear convolution, the time-domain signal overlaps with itself.

- Under-sampling $\rightarrow$ Frequency Overlap (Spectral Aliasing)
 - Under-sizing the FFT $\rightarrow$ Time Overlap (Circular/Time Aliasing)
-

4. OVERLAP-SAVE VISUAL WALKTHROUGH

Instead of heavy matrix math, let's look at the purely visual, diagrammatic flow of Overlap-Save.

Step 1: Overlapping the Input Blocks

We take blocks of length M . Unlike *Overlap-Add* (where blocks sit side-by-side), *Overlap-Save* blocks physically overlap by $M - 1$ samples. The first $M - 1$ samples of **Block 2** are simply a copy of the last $M - 1$ samples of **Block 1**.

Step 2: Circular Convolution (The FFT Magic)

We take the FFT of the block, multiply it by the FFT of the filter, and take the Inverse FFT. Because it's circular, the end of the signal wraps around and mathematically mangles the beginning of the block.

Step 3: The Trash Can (Discarding the Garbage)

We look at the resulting block of length M . We take a pair of scissors and cut off the first $M - 1$ samples. They are time-aliased garbage. We throw them away.

Step 4: Direct Concatenation (Saving)

We take the remaining $N - M + 1$ good samples (where $N = N_{\text{total}}$) and place them directly into our output stream. **No addition is required!** We just snap the saved blocks together like Lego bricks.

Pictorial Representation

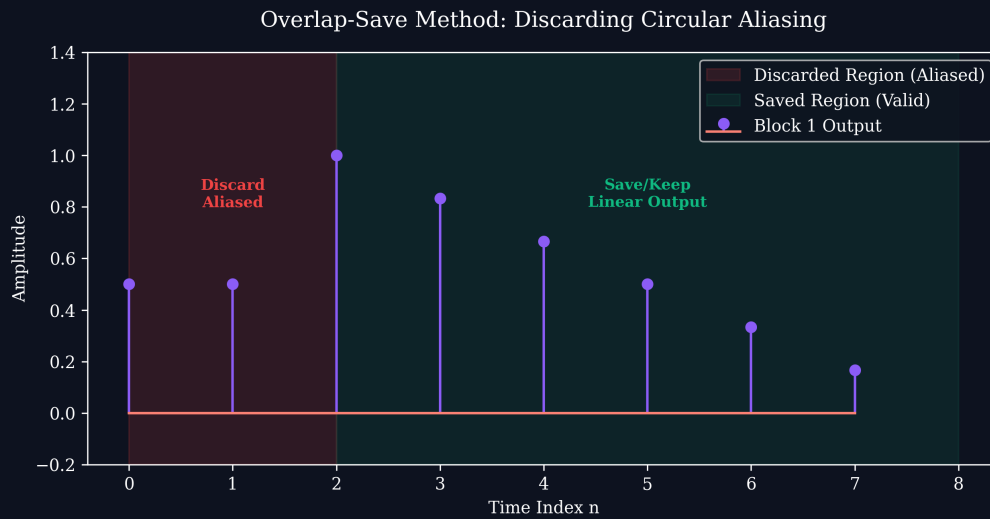


Figure 1: Overlap Save Process

(Notice how the overlapped input data leads to discarded garbage at the front of every output block, leaving perfectly seamless output data).

5. SUMMARY: OVERLAP-ADD VS. OVERLAP-SAVE

— Feature — Overlap-Add (OLA) — Overlap-Save (OLS) — — : — — : — — — **Input Blocks** — Non-overlapping — Overlapping by -1 samples || **Zero Padding** | Yes (Padded with -1 zeros) — No — — **FFT Effect** — Tails expand into the zero-padding — Wrap-around corrupts the head — — **Output Assembly** — Must **ADD** overlapping tails — **DISCARD** heads, directly concatenate — — **Hardware Benefit** — Simple input slicing — Extremely fast output (No adders needed) —